

ELEC0520

Python programming

Summer 2026

list, range(), str-split(), collection, iterable

https://colab.research.google.com/drive/1BxvFFHFsSUpBtA-Kg-pmKmb9rixJjIE_?usp=sharing

리스트list

리스트의 필요성

- 유사한 특성을 가진 변수가 여러 개 필요한 경우

대화창 실습 : 여러 개의 변수를 생성하고 사용하기

```
>>> score0 = 87
```

```
>>> score1 = 84
```

```
>>> score2 = 95
```

```
>>> score3 = 67
```

```
>>> score4 = 88
```

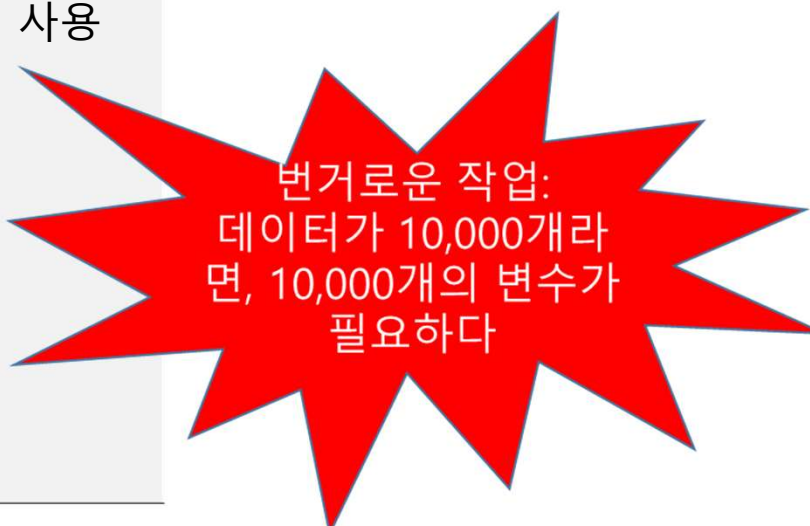
```
>>> score5 = 94
```

```
>>> score6 = 63
```

```
>>> print(score0, score3)
```

```
87 67
```

데이터가 7개 있기 때문에 7개의 개별적인
변수를 선언하여 이 변수에 값을 담아서 사용



번거로운 작업:
데이터가 10,000개라
면, 10,000개의 변수가
필요하다

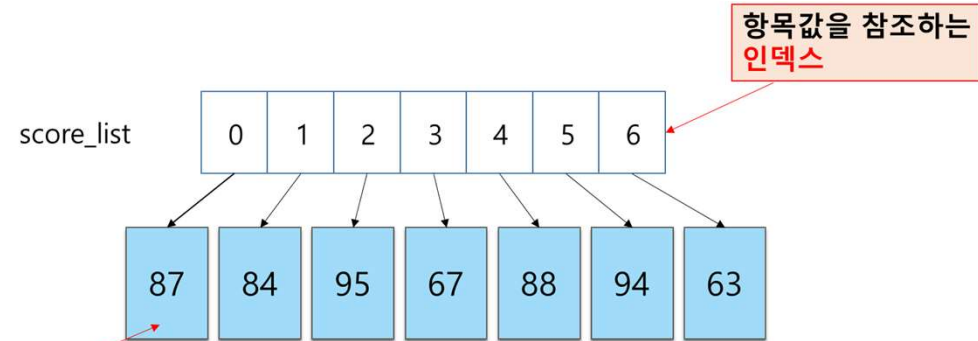
리스트 list

- 개별적인 값을 하나의 변수에 담아서 처리
 - 매번 변수의 이름을 작성하고 관리하는 것보다 편리하고 효율적
 - 한꺼번에 복사 및 조작 가능
- 리스트
 - 여러 개의 항목을 포함할 수 있는 파이썬 자료구조
 - 내부 항목 교체 가능
 - 저장된 항목들 중 필요한 값을 추출 가능

리스트의 성분

- 항목 **item** 또는 성분, 요소 **element**

- 리스트를 이루는 원소로 심표로 구분된 자료 값
- 대괄호 [] 내에 심표를 이용하여 값을 구분



대화창 실습 : 수치 값을 가진 리스트 만들기

```
>>> score_list = [87, 84, 95, 67, 88, 94, 63]
```

```
>>> score_list
```

```
[87, 84, 95, 67, 88, 94, 63]
```

```
>>> print(score_list[0], score_list[3])
```

```
87 67
```

요소의 참조는 리스트 이름과 인덱스를 사용한다

리스트 성분의 자료형

- fruits 리스트에 'banana', 'apple', 'orange', 'kiwi'의 4개 문자열 값을 한꺼번에 저장하고 출력
- mixed_list에 100, 200, 400과 같은 정수 값과 'apple'이라는 문자열 값을 동시에 리스트에 저장하고 출력

대화창 실습 : 문자열과 복합 자료값을 가진 리스트 만들기

```
>>> fruits = ['banana', 'apple', 'orange', 'kiwi'] # 문자열을 가지는 리스트
```

```
>>> fruits
```

```
['banana', 'apple', 'orange', 'kiwi']
```

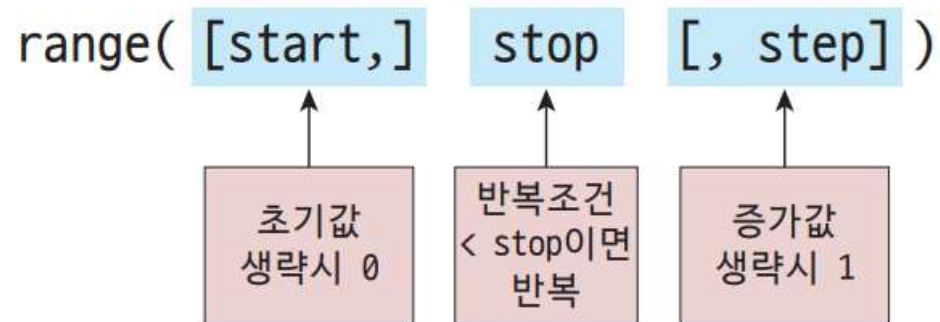
```
>>> mixed_list = [100, 200, 'apple', 400]
```

```
>>> mixed_list
```

```
[100, 200, 'apple', 400]
```

파이썬의 리스트는 정수, 실수, 문자열 등 서로 다른 자료형을 가질 수 있다

range() 함수



대화창 실습 : range() 함수의 활용과 리스트

```
>>> list(range(5))
```

```
>>> list(range(0, 5))
```

```
>>> list(range(0, 5, 1))
```

```
>>> list(range(0, 5, 2))
```

```
>>> list(range(2, 5))
```

```
>>> list(range(0, 10, 2))
```

```
>>> list(range(1, 10, 2))
```

```
>>> list(range(-2, -10, -2))
```

range()나 문자열을 이용하여 리스트 생성

- range(1, 10)이라는 함수를 통해 1부터 9까지의 숫자의 **열(sequence)**을 얻은 후 이 열을 원소로 가지는 리스트를 list() 함수를 통해 생성

대화창 실습 : 다양한 방법으로 리스트 만들기

```
>>> list1 = list()           # 빈 리스트 생성하기 1
>>> list2 = []              # 빈 리스트 생성하기 2
>>> list3 = list((1, 2, 3))  # 튜플로부터 리스트 생성
>>> list3
[1, 2, 3]
>>> list4 = list(range(1, 10)) # range() 함수로부터 리스트 생성
>>> list4
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list5 = list('ABCDEF')    # 문자열로부터 리스트 생성
>>> list5
['A', 'B', 'C', 'D', 'E', 'F']
```




LAB 6-1: 리스트의 생성

1. 1부터 10까지의 숫자 중에서 짝수를 요소로 가지는 `even_list`라는 리스트를 생성하여라(10을 포함). `print()` 함수를 사용하여 이 리스트를 다음과 같이 출력하여라.

```
even_list = [2, 4, 6, 8, 10]
```

2. `range()` 함수를 이용하여 1번의 문제를 다시 풀어보시오.

3. 'Korea', 'China', 'India', 'Nepal'의 네 원소를 가지는 `nations`라는 리스트를 생성하여라. `print()` 함수를 사용하여 이 리스트를 다음과 같이 출력하여라.

```
nations = ['Korea', 'China', 'India', 'Nepal']
```

4. 여러분의 친한 친구 5명의 이름을 원소로 가지는 `friends`라는 리스트를 생성하여라. 그리고 다음과 같이 출력하여라.

```
friends = ['길동', '철수', '은지', '지은', '영민']
```

5. 'XYZ' 문자열을 이용하여 'X', 'Y', 'Z'라는 요소를 가지는 `string`이라는 이름의 리스트를 생성하고 다음과 같이 출력하여라.

```
string = ['X', 'Y', 'Z']
```

리스트 항목 수정: 리스트 축약 사용



코드 6-9: 리스트의 축약 표현을 사용하여 10을 곱하는 기능

`list_element_ex2.py`

고급 기능으로 11장에서 상세히 다룹니다

```
list1 = [10, 20, 30, 40, 50]
list1 = [n * 10 for n in list1] # 리스트 축약 표현을 사용
print(list1)
```

실행결과

```
[100, 200, 300, 400, 500]
```

리스트 생성 3가지 방법

1. 성분 나열

2. 캐스팅(casting)

- str을 list로 캐스팅
- 비교) str을 split()

```
>>> st = 'abc def'
>>> list(st)
['a', 'b', 'c', ' ', 'd', 'e', 'f']
>>> st.split()
['abc', 'def']
```

3. 리스트 축약(list comprehension)

- [모을성분 for 성분 in 어디에서 if 조건]

인덱싱indexing

인덱스index

리스트의 항목 값을 가리키는 숫자

n개의 항목을 가진 리스트의 인덱스는 0부터 n-1 까지의 값을 가짐

인덱싱indexing

항목의 인덱스를 이용하여 자료 값에 접근

대화창 실습 : 리스트의 인덱스 사용법

```
>>> n_list = [11, 22, 33, 44, 55, 66]
```

```
>>> n_list
```

```
[11, 22, 33, 44, 55, 66]
```

```
>>> len(n_list)
```

```
>>> n_list[0]
```

```
>>> n_list[1]
```

```
>>> n_list[5]
```

```
>>> n_list[6]
```

```
...
```

```
IndexError: list index out of range
```

```
>>> n_list[-1]
```

```
>>> n_list[-2]
```

```
>>> n_list[-3]
```

음수 index

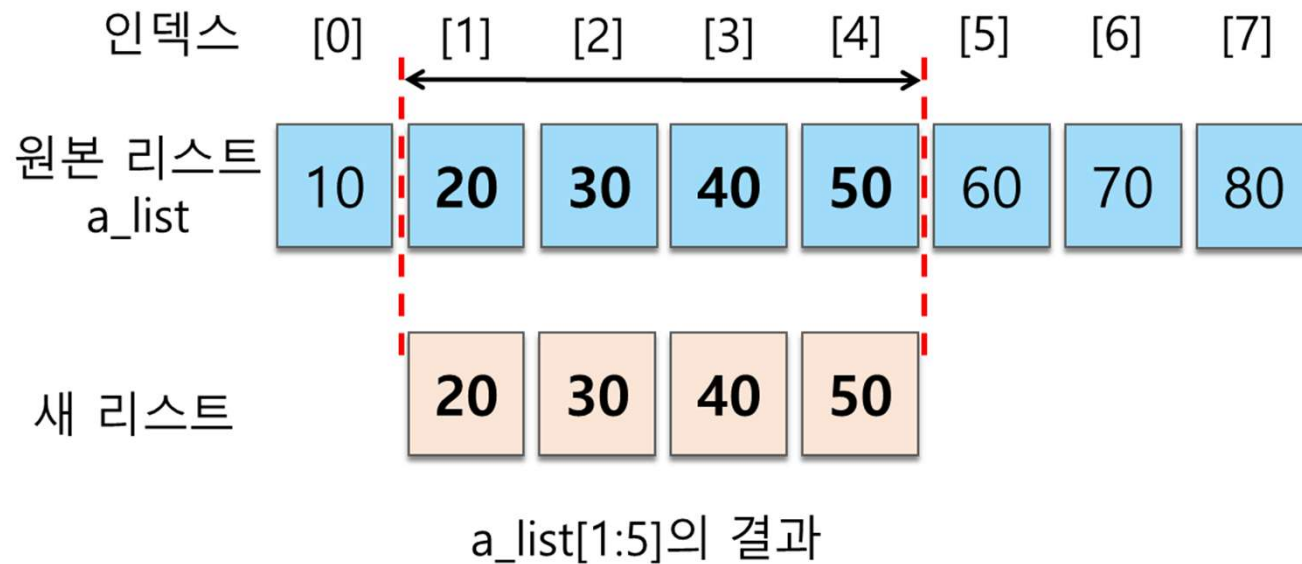
n_list =

[-6]	[-5]	[-4]	[-3]	[-2]	[-1]
11	22	33	44	55	66

슬라이싱slicing

슬라이싱

- 리스트내의 항목을 특정한 구간별로 선택하여 잘라내는 기능
- 구간을 명시하기 위해 **리스트_이름[start : end]**



슬라이싱 범위 및 스텝 지정

문법	하는 일
<code>a_list[start:end]</code>	start부터 (end-1)까지의 항목들을 슬라이싱(end 인덱스의 항목은 포함하지 않음)
<code>a_list[start:]</code>	start부터 리스트의 끝까지, 즉 뒷부분 모두를 슬라이싱
<code>a_list[:end]</code>	처음부터 end-1번째 인덱스 항목을 슬라이싱
<code>a_list[:]</code>	전체를 슬라이싱
<code>a_list[start:end:step]</code>	start부터 end-1까지를 step만큼 건너뛰며 슬라이싱
<code>a_list[-2:]</code>	
<code>a_list[:-2]</code>	
<code>a_list[::-1]</code>	
<code>a_list[1::-1]</code>	

대화창 실습 : 리스트와 슬라이싱

```
>>> a_list = [10, 20, 30, 40, 50, 60, 70, 80]
```

```
>>> a_list[1:5]
```

```
>>> a_list[0:1]
```

```
>>> a_list[:2]
```

```
>>> a_list[1:]
```

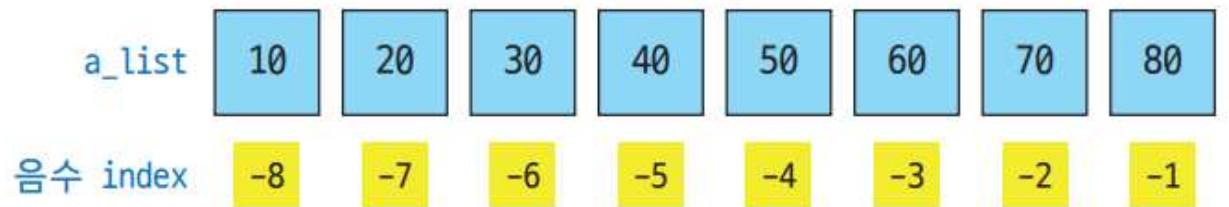
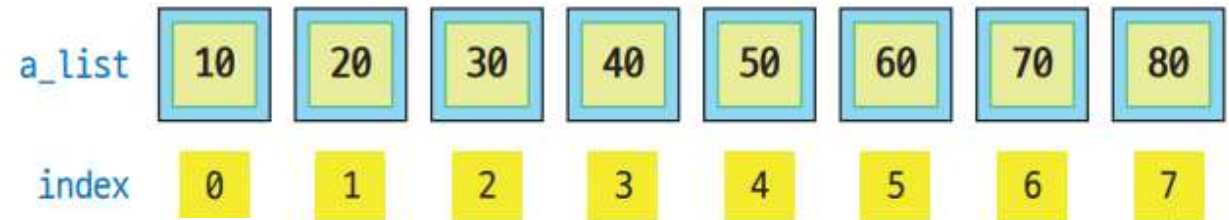
```
>>> a_list[:]
```

```
>>> a_list[-7:-2]
```

```
>>> a_list[-7:]
```

```
>>> a_list[:-2]
```

```
>>> a_list[-2:]
```



[그림 5-9] a_list와 음수 인덱스

리스트에 사용 가능한 내장함수

built-in ft

len(), **sum()**, **max()**, **min()**, **sorted()**

- **sum()** – 리스트의 모든 성분이 숫자일때만 가능
 - `sum([1,2,'a'])`는 오류발생
- **max()**, **min()**, **sorted()** – 리스트 성분의 자료형이 일치할때만 가능
 - `sorted(['a', 'b'])`는 가능
 - `sorted(['a', 'b', 3])`은 오류 발생

대화창 실습 : 리스트와 내장함수 min(), max(), sum()

```
>>> list1 = [20, 10, 40, 50, 30]
>>> min(list1)  # 리스트의 원소들 중 가장 작은 원소를 구한다.
10
>>> max(list1)  # 리스트의 원소들 중 가장 큰 원소를 구한다.
50
>>> sum(list1)  # 리스트내의 원소의 합을 구한다.
150
>>> len(list1)  # 리스트내의 항목 개수를 구한다.
5

>>> fruits = ['banana', 'orange', 'apple', 'kiwi']
>>> min(fruits)  # 영어사전 순서로 가장 앞에 있는 단어를 반환
'apple'
>>> max(fruits)  # 영어사전 순서로 가장 뒤에 있는 단어를 반환
'orange'

#sum() 함수는 문자열에 대해서는 동작하지 않음
```

리스트에 적용되는 내장함수

- min(), max()
- sum()
- len()
- sorted()



LAB 6-4: 리스트의 min()과 max(), sum(), len() 함수

1. 1부터 10까지의 수 중에서 소수 원소를 가지는 prime_list라는 리스트를 생성하고 이들 중 최솟값, 최댓값을 다음과 같이 출력하시오. min()과 max(), sum(), len() 내장함수를 사용하여 다음과 같이 출력하시오.

1에서 10까지의 소수 : [2, 3, 5, 7]

최솟값 : 2

최댓값 : 7

합계 : 17

평균 : 4.25

2. 'Korea', 'China', 'Russia', 'Malaysia' 원소를 가지는 nations라는 리스트가 있다. 이들 나라 중에서 사전 순서로 가장 먼저 나오는 나라와 가장 뒤에 나오는 나라를 다음과 같이 출력하시오.

국가 목록 : ['Korea', 'China', 'Russia', 'Malaysia']

사전에 가장 먼저 나오는 나라 : China

사전에 가장 뒤에 나오는 나라 : Russia

리스트와 연산

in, not in

+, *

==, !=, >, >=, <, <=

멤버 연산자: in, not in

- 멤버 연산자는 참(True) 혹은 거짓(False)를 반환하는 연산자
- 특정한 원소가 문자열, 리스트, 튜플과 같은 자료구조의 내부에 포함되어 있는가를 검사하는 용도로 사용됨



코드 6-4: 리스트 내부에 값이 존재하는가를 확인하는 기능
value_in_list.py

```
n_list = [11, 22, 33, 44, 55, 66]

print(88 in n_list) # 88은 n_list에 없음
print(55 in n_list) # 55는 n_list에 있음
```

in 연산으로 존재여부를 확인하고 True인 경우에만 삭제할 경우는 오류가 생기지 않음

실행결과

```
False
True
```

+



코드 6-6: 두 리스트를 합치는 연산자 +를 이용하여 그 결과를 저장함
list_plus_ex.py

```
list1 = [11, 22, 33, 44]
list2 = [55, 66]

list3 = list1 + list2  # 리스트끼리의 덧셈 연산은 가능하다.
print(list3)
```

실행결과

```
[11, 22, 33, 44, 55, 66]
```

*



코드 6-7: 리스트에 대한 정수 곱셈 연산의 결과

list_mult_ex.py

```
list1 = [1, 2, 3, 4]
print('list1 * 2 =', list1 * 2)
print('list1 * 3 =', list1 * 3)
```

실행결과

```
list1 * 2 = [1, 2, 3, 4, 1, 2, 3, 4]
list1 * 3 = [1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4]
```



대화창 실습: 문법 오류를 유발하는 두 리스트의 * 연산

```
>>> list1 = [1, 2, 3, 4]
>>> list2 = [10, 20, 30]
>>> list1 * list2    # 리스트끼리의 곱셈은 불가능하다.
...
TypeError: can't multiply sequence by non-int of type 'list'
```


= **=**



대화창 실습: == 연산자를 이용한 리스트의 비교

```
>>> list1 = [1, 2, 3, 4]
>>> list2 = [1, 2, 3, 4]
>>> list1 == list2    # 두 리스트의 내용이 같은지 비교한다.
True
>>> list3 = [4, 1, 2, 3]
>>> list1 == list3    # 두 리스트의 내용이 같은지 비교한다.
False
```

>, >=, <, <=

- 문자열의 경우 사전적 순서 **lexicographic order**를 비교



대화창 실습: >, < 연산자를 이용한 리스트의 비교

```
>>> list1 = [1, 2, 3, 4]
>>> list2 = [2, 3, 3, 4]
>>> list1 > list2
False
>>> list1 < list2
True
```

컬렉션collection

컬렉션 **collection**

- 여러 개의 요소(성분, 원소)를 포함하는 자료형
 - 여러 성분을 각기 다른 문자로 감싼다.
 - str: 따옴표. *e.g.* 'hello'
 - list: [] *e.g.* [1, 2, 3]
 - tuple: () *e.g.* (1, 2, 3)
 - set: {} *e.g.* {1, 2, 3}
 - dict: {} *e.g.* {'kor': 95, 'eng': 98, 'math': 100}
 - range: range(), *e.g.* range(1, 11)

Built-in functions

- 컬렉션에 다양한 내장 함수, 연산을 할 수 있다.
 - len()
 - in 연산자
- 이터러블(Iterable)에 사용할 수 있는 함수
 - sum(): 컬렉션의 모든 성분이 숫자인 경우에만
 - sorted()
 - max()/min()
- iterable?
 - str, list, tuple, dict, set, range, ... 은 collection이자 iterable

Built-in functions(cont.)

- built-in ft. `len()`, `max()`, `min()`, `sorted()`은 iterable에 사용할 수 있으므로 당연히 문자열(str)에도 가능

```
>>> len(st)
13
>>> max(st)
'w'
>>> min(st)
' '
```

- 인덱싱과 슬라이싱도 문자열에 가능

```
>>> st = 'hello, world!'
>>> st[-1]

>>> st[0]

>>> st[:5]

>>> st[7:-1]

>>> st[7:12]
```

문자열 메소드 1 **method**

문자열의 메소드 - split()

- 메소드

- 특정 객체에서 사용할 수 있는 함수
- 내장 함수와 달리 객체에서 도트연산자를 작성한 후에 호출 가능

- 문자열의 split() 메소드

- 문자열을 특정 구분자로 분리하는 메소드
- split()의 인자는 구분자 **separator**
 - 생략할 경우 공백으로 구분

split() 메소드 예시

구분자 **separator**

한 문자열을 하나 이상의 개별 문자열로 나누는 문자

대화창 실습 : 여러가지 문자열 메소드

>>> 'Guido van Rossum'.split()

['Guido', 'van', 'Rossum']

>>> 'Apple,Banana,Orange'.split(',')

['Apple', 'Banana', 'Orange']

>>> 'Apple|Banana|Orange|Kiwi'.split('|')

['Apple', 'Banana', 'Orange', 'Kiwi']

디폴트 구분자는 공백

구분자로 쉼표를 사용

구분자로 세로바를 사용

유용한 내장함수들

dir()

- 해당 자료, 자료형에서 사용할 수 있는 속성 리스트 반환
- 해당 클래스(자료형)에서 사용할 수 있는 속성(메소드, 멤버 변수) 리스트 반환

```
>>> dir(str)
['__add__', '__class__', '__contains__', '__delattr__', '__dict__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__getitem__', '__getnewargs__', '__getstate__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmul__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'hex', 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'translate', 'upper', 'zfill']
```

help()

- 도움말 반환
- 객체 도움말 반환

```
>>> help(sorted)
Help on built-in function sorted in module builtins:

sorted(iterable, /, *, key=None, reverse=False)
    Return a new list containing all items from the iterable in ascending order.

    A custom key function can be supplied to customize the sort order,
    and the
    reverse flag can be set to request the result in descending order.

>>> help(sum)
Help on built-in function sum in module builtins:

sum(iterable, /, start=0)
    Return the sum of a 'start' value (default: 0) plus an iterable of numbers

    When the iterable is empty, return the start value.
    This function is intended specifically for use with numeric values
    and may
    reject non-numeric types.

>>> help(max)
Help on built-in function max in module builtins:

max(...)
    max(iterable, *[, default=obj, key=func]) -> value
    max(arg1, arg2, *args, *[, key=func]) -> value

    With a single iterable argument, return its biggest item. The
    default keyword-only argument specifies an object to return if
```

help()

- 함수가 아닌 메소드라면?
 - help(자료형.메소드)

```
>>> help(str.split)
Help on method_descriptor:

split(self, /, sep=None, maxsplit=-1)
    Return a list of the substrings in the string, using sep as the separator string.
```

type()

- 자료형을 반환
- 해당 객체가 생성된 클래스명을 반환하는 함수

```
>>> a = 'abc'
>>> type(a)
<class 'str'>
>>> a = 3
>>> type(a)
<class 'int'>
>>> a = 3.1
>>> type(a)
<class 'float'>
>>> a = [1,2,3]
>>> type(a)
<class 'list'>
>>> a = range(5)
>>> type(a)
<class 'range'>
```